🚢 Three-Tier K8s Portfolio Deployment Guide

Welcome to the Kubernetes Deployment guide for the DevOps Portfolio project! This document is designed for students to understand how a classic **Three-Tier Architecture** is containerized and deployed into a Kubernetes cluster.

🏗️ Architecture Overview

This project consists of three distinct components (tiers) communicating with each other:

1. **Frontend (React + Nginx)**
   - A modern UI built with React and styled with TailwindCSS.
   - Served via **Nginx**, which also acts as a **Reverse Proxy**, forwarding `/api/` calls directly to the Backend service.
2. **Backend (Spring Boot + Java)**
   - A secure RESTful API that handles CRUD operations (Create, Read, Update, Delete) for the portfolio data (skills, projects, experiences).
3. **Database (MySQL)**
   - A relational database that persistently stores all data.
   - Deployed as a **StatefulSet** with persistent volumes to ensure data survives pod restarts.

🛠️ Prerequisites

Before you start deploying, ensure you have the following tools installed and running:

- **Docker Desktop** (or equivalent container runtime)
- **kubectl** (Kubernetes command-line tool)
- **Minikube** (for local Kubernetes testing)

Start your Minikube cluster:

```
minikube start
```

🚀 Deployment Steps (The "Kubernetes Way")

In Kubernetes, we must respect **dependencies**. The Frontend depends on the Backend, and the Backend depends on the Database. Therefore, we deploy from the **bottom up**.

Step 1: Deploy the Database (MySQL)

We start with the database so that when the backend spins up, it has a database ready to connect to.

```
# 1. Create the MySQL credentials secret
kubectl apply -f k8s/database/secret.yml

# 2. Apply ConfigMap containing initialization scripts/schemas
kubectl apply -f k8s/database/config.yml

# 3. Deploy the StatefulSet and its Headless Service
kubectl apply -f k8s/database/stateful-set.yml
kubectl apply -f k8s/database/service.yml
```

> **Teaching Point:** We use a `StatefulSet` rather than a standard `Deployment` for the database because databases need stable, persistent storage and predictable network identities.

Step 2: Deploy the Backend (Spring Boot API)

Now that the DB is running, we deploy the Spring Boot backend.

```
# 1. Deploy the Backend application
kubectl apply -f k8s/backend/deployment.yml

# 2. Expose the Backend via a Service
kubectl apply -f k8s/backend/service.yml
```

> **Teaching Point:** The backend uses environment variables injected via Kubernetes (e.g., `DB_HOST`, `DB_USER`) to dynamically find the database service across the cluster internal network (using Kubernetes DNS, e.g. `portfolio-mysql-0.mysql:3306`).

Step 3: Deploy the Frontend (React + Nginx)

Finally, we deploy the frontend UI.

```
# 1. Apply the Nginx Reverse Proxy Configuration
kubectl apply -f k8s/frontend/config.yml

# 2. Deploy the Frontend Application
kubectl apply -f k8s/frontend/deployment.yml

# 3. Expose the Frontend via a NodePort Service
kubectl apply -f k8s/frontend/service-nodeport.yml
```

> **Teaching Point:** Notice the `config.yml` creates a `ConfigMap` for `nginx.conf`. This configuration tells Nginx to serve the static React files on port 80, but if a request starts with `/api/`, it routes it to the backend (`http://portfolio-backend-service:8080`).

✅ Verifying the Deployment

Check if all your pods are healthy and running:

```
kubectl get pods
```

Expected Output:

```
NAME                                READY   STATUS    RESTARTS   AGE
portfolio-backend-xxxxxxxxx-xxxxx   1/1     Running   0          2m
portfolio-frontend-xxxxxxxxx-xxxxx  1/1     Running   0          1m
portfolio-mysql-0                   1/1     Running   0          5m
```

🌐 Accessing the Application

Because we are running locally in Minikube, our cluster operates in a virtual machine. To expose our `NodePort` (port 30082) so our host browser can reach it, we use the `minikube service` command:

```
minikube service portfolio-frontend-service --url
```

Click the generated link (e.g., `http://127.0.0.1:49818`) to view the beautiful, animated DevOps Portfolio!

🔄 Updating / Rebuilding Images (For Developers)

If you modify the Java or React code and wish to build new images for your cluster, a `build.sh` script is provided to handle multi-architecture Docker building (crucial for teams with a mix of Intel/AMD and Apple Silicon/ARM processors).

```
# Run the build tool (Make sure Docker is running!)
./build.sh

# Afterwards, restart your deployments to pull the fresh images
kubectl rollout restart deployment/portfolio-backend
kubectl rollout restart deployment/portfolio-frontend
```

Happy Learning & Happy Deploying! 🚢